SENATI
Curso: Inteligencia Artificial

Guía 03

Uso de las librerías SciPy y NLTK

Cálculo científico y procesamiento de lenguaje natural en Visual Studio Code

Dato	Detalle
Instructor	Rembrandt Ubalde Enríquez
Estudiante	_____
Fecha	____ / ____ / 2026
Duración estimada	5 horas pedagógicas
Herramientas	Python 3, Visual Studio Code, SciPy, NLTK, NumPy

Guía práctica paso a paso con Visual Studio Code

Contenido

- Presentación de la guía
- Parte 1. Preparación del entorno en Visual Studio Code
- Parte 2. Cálculo científico con SciPy
- Parte 3. Procesamiento de lenguaje natural con NLTK
- Parte 4. Proyecto integrador: SciPy + NLTK
- Parte 5. Errores frecuentes y cómo resolverlos
- Parte 6. Ejercicios propuestos
- Lista de cotejo (autoevaluación)

Presentación de la guía

Objetivo general

Estudiar el uso de la librería **SciPy** para resolver problemas de estadística, optimización y cálculo, y de la librería **NLTK** para procesar textos en español, integrando ambas en un pequeño proyecto de inteligencia artificial en Visual Studio Code.

Objetivos específicos

- Preparar un proyecto en VS Code con SciPy y NLTK, y descargar los recursos de NLTK.
- Aplicar estadística descriptiva, pruebas de hipótesis, correlación y distribución normal con `scipy.stats`.
- Resolver problemas de optimización, ajuste de curvas, integración, sistemas de ecuaciones e interpolación.
- Tokenizar, normalizar, quitar palabras vacías, obtener raíces y contar palabras con NLTK.
- Entrenar un clasificador de opiniones (positivo/negativo) con NLTK.
- Calcular la similitud entre textos combinando NLTK y SciPy.

¿Qué es cada librería?

Librería	Descripción	Aplicación en IA
SciPy	Librería de cálculo científico construida sobre NumPy. Se organiza en submódulos: `stats`, `optimize`, `integrate`, `linalg`, `interpolate`, `spatial`, entre otros.	Analizar datos antes de entrenar, comparar resultados de modelos, optimizar funciones y medir distancias entre vectores.
NLTK (Natural Language Toolkit)	Librería para procesamiento de lenguaje natural (PLN): tokenización, palabras vacías, stemming, frecuencias, n-gramas y clasificadores.	Preparar textos para chatbots, análisis de sentimiento, clasificación de documentos y buscadores.

Requisitos previos

- Haber completado la **Guía 01 (NumPy y Pandas)**.
- Conexión a Internet para descargar los recursos de NLTK la primera vez.

Parte 1. Preparación del entorno en Visual Studio Code

En esta parte dejarás listo tu equipo para programar en Python dentro de Visual Studio Code (VS Code). Si ya lo hiciste en una guía anterior, revisa rápidamente cada paso y continúa desde el **Paso 4** creando una carpeta nueva para esta guía.

Paso 1. Instalar Python

1. Ingresa a **python.org/downloads** y descarga la versión estable más reciente de Python 3 (3.11 o superior).
2. Ejecuta el instalador. En la primera pantalla **marca la casilla "Add python.exe to PATH"** y luego pulsa **Install Now**.
3. Al terminar, abre el **Símbolo del sistema** (cmd) y verifica la instalación con los comandos de abajo.

Símbolo del sistema (Windows)

```
python --version
pip --version
```

Resultado esperado en la terminal

```
Python 3.12.x
pip 24.x from C:\Users\...\site-packages\pip (python 3.12)
```

Si el comando python no funciona

Mensaje "No se encontró Python; ejecutar sin argumentos para instalar desde Microsoft Store...": Python no está instalado o no está en el PATH, y Windows redirige el comando a la Store. Para solucionarlo:
1. Instala Python desde **python.org** marcando **Add python.exe to PATH**. Sin permisos de administrador, desmarca **Use admin privileges when installing py.exe**. También puedes usar en la terminal: `winget install Python.Python.3.12`
2. Abre **Configuración > Aplicaciones > Configuración avanzada de aplicaciones > Alias de ejecución de aplicaciones** y desactiva **python.exe** y **python3.exe** (Instalador de aplicación).
3. **Cierra VS Code por completo y vuelve a abrirlo**: una terminal ya abierta conserva el PATH anterior.
4. Verifica con `python --version` o con `py --version`.

Mensaje "python no se reconoce como un comando interno": Python se instaló sin agregarse al PATH. Vuelve a ejecutar el instalador, elige **Modify** y activa **Add Python to environment variables**.

Paso 2. Instalar Visual Studio Code y sus extensiones

1. Descarga VS Code desde **code.visualstudio.com** e instálalo. Durante la instalación marca **"Agregar la acción Abrir con Code"** y **"Agregar a PATH"**.
2. Abre VS Code y entra al panel de **Extensiones** (ícono de cuadritos en la barra izquierda o `Ctrl + Shift + X`).
3. Busca e instala la extensión **Python** (publicada por Microsoft). Se instalará también **Pylance**, que da autocompletado y detección de errores.
4. Instala la extensión **Jupyter** (Microsoft) para trabajar con cuadernos `.ipynb` dentro de VS Code.

5. Opcional: instala **Spanish Language Pack for Visual Studio Code** si prefieres los menús en español.

Paso 3. Conocer la interfaz de VS Code

Zona de VS Code	Para qué sirve
Explorador (`Ctrl + Shift + E`)	Muestra las carpetas y archivos del proyecto. Desde aquí creas archivos nuevos.
Editor	Zona central donde escribes el código. Puedes tener varias pestañas abiertas.
Terminal integrada (`Ctrl + ñ` o menú **Ver > Terminal**)	Ejecuta comandos (pip, python) sin salir de VS Code.
Paleta de comandos (`Ctrl + Shift + P`)	Busca cualquier acción de VS Code escribiendo su nombre.
Barra de estado (abajo)	Muestra el intérprete de Python activo, errores y advertencias.
Botón ▶ Run Python File (arriba a la derecha)	Ejecuta el archivo `.py` abierto y muestra el resultado en la terminal.

Paso 4. Crear la carpeta del proyecto

1. En el Explorador de Windows crea la carpeta **IA_Guia03_SciPy_NLTK** (por ejemplo, en Documentos).
2. En VS Code ve a **Archivo > Abrir carpeta...** y selecciona la carpeta creada.
3. Si aparece el mensaje **"¿Confía en los autores de los archivos de esta carpeta?"**, pulsa **Sí, confío en los autores**.

Paso 5. Crear y activar un entorno virtual

Un **entorno virtual** es una instalación de Python aislada solo para este proyecto. Evita conflictos entre versiones de librerías de distintos proyectos.

1. Abre la terminal integrada con **Ver > Terminal** (o `Ctrl + ñ`).
2. Escribe el comando para crear el entorno virtual llamado `venv`.
3. Actívalo. Sabrás que está activo porque al inicio de la línea aparece **(venv)**.

Terminal de VS Code

```
# Crear el entorno virtual
python -m venv venv

# Activarlo en Windows (PowerShell)
.\venv\Scripts\Activate.ps1

# Activarlo en Windows (cmd)
venv\Scripts\activate

# Activarlo en Linux / macOS
```

```
source venv/bin/activate
```

> **Importante**
> Si PowerShell muestra **"la ejecución de scripts está deshabilitada en este sistema"**, ejecuta una sola vez:
> `Set-ExecutionPolicy -Scope CurrentUser -ExecutionPolicy RemoteSigned` y responde **S** (Sí). Luego vuelve a activar el entorno.

Paso 6. Seleccionar el intérprete de Python en VS Code

1. Presiona `Ctrl + Shift + P` y escribe **Python: Select Interpreter**.
2. Elige la opción que contiene **('venv': venv)** y la ruta `.\venv\Scripts\python.exe`.
3. Comprueba en la barra de estado (parte inferior derecha) que aparezca la versión de Python con **('venv')**.

Paso 7. Instalar las librerías de la guía

Con el entorno virtual activo, instala las librerías desde la terminal integrada:

Terminal de VS Code

```
python -m pip install --upgrade pip
pip install numpy scipy nltk matplotlib
```

Verifica que la instalación fue correcta:

Terminal de VS Code

```
python -c "import scipy, nltk; print(scipy.__version__, nltk.__version__)"
```

> **Sugerencia**
> Guarda la lista de librerías de tu proyecto con `pip freeze > requirements.txt`. Así un compañero puede instalar lo mismo con `pip install -r requirements.txt`.

Paso 8. Cómo crear y ejecutar cada archivo de esta guía

1. En el Explorador de VS Code pulsa el ícono **Nuevo archivo** (o `Ctrl + N` y luego `Ctrl + S`).
2. Escribe el nombre indicado en la guía con extensión `.py` (por ejemplo `01_ejemplo.py`) y presiona Enter.
3. Copia el código, guárdalo con `Ctrl + S` y ejecútalo con el botón ▶ **Run Python File** (arriba a la derecha) o escribiendo `python nombre_archivo.py` en la terminal.
4. Compara lo que aparece en la terminal con el recuadro verde **"Resultado esperado"**.
5. Para ejecutar solo algunas líneas, selecciónalas y presiona `Shift + Enter`: se ejecutarán en la ventana interactiva de Python.

Parte 2. Cálculo científico con SciPy

Submódulo	Para qué sirve	Funciones de esta guía
`scipy.stats`	Estadística y probabilidad	`describe`, `mode`, `ttest_ind`, `pearsonr`, `norm`
`scipy.optimize`	Encontrar mínimos y ajustar curvas	`minimize_scalar`, `curve_fit`
`scipy.integrate`	Integración numérica	`quad`
`scipy.linalg`	Álgebra lineal	`solve`, `det`
`scipy.interpolate`	Estimar valores intermedios	`CubicSpline`
`scipy.spatial`	Distancias entre vectores	`distance.cosine`

Paso 9. Estadística con scipy.stats

Crea el archivo **01_scipy_estadistica.py**. Compararemos las notas de dos secciones y analizaremos la relación entre horas de estudio y notas.

Archivo: 01_scipy_estadistica.py

```
# 01_scipy_estadistica.py - Estadística con scipy.stats
import numpy as np
import scipy
from scipy import stats

print("Versión de SciPy:", scipy.__version__)

# Notas de dos secciones de un mismo curso
seccion_a = np.array([14, 16, 13, 15, 17, 12, 15, 16, 14, 18])
seccion_b = np.array([12, 13, 11, 14, 12, 10, 13, 15, 11, 12])

# 1. Resumen estadístico
resumen = stats.describe(seccion_a)
print("Media A:", round(resumen.mean, 2), "| Varianza A:", round(resumen.variance, 2))
print("Moda de B:", stats.mode(seccion_b).mode)

# 2. Prueba t: ¿las dos secciones tienen promedios distintos?
t, p = stats.ttest_ind(seccion_a, seccion_b)
print(f"Estadístico t = {t:.3f} | valor p = {p:.4f}")
if p < 0.05:
    print("Conclusión: la diferencia de promedios es significativa (p < 0.05)")
else:
    print("Conclusión: no hay diferencia significativa")

# 3. Correlación de Pearson: horas de estudio vs. nota
horas = np.array([2, 3, 5, 6, 8, 9, 10])
notas = np.array([10, 11, 13, 14, 16, 17, 19])
r, p_r = stats.pearsonr(horas, notas)
print(f"Correlación r = {r:.3f} | valor p = {p_r:.6f}")

# 4. Distribución normal
normal = stats.norm(loc=14, scale=2)      # media 14, desviación 2
print("P(nota <= 12):", round(normal.cdf(12), 4))
print("P(nota >= 17):", round(1 - normal.cdf(17), 4))
```

Resultado esperado en la terminal

```
Versión de SciPy: 1.18.1
Media A: 15.0 | Varianza A: 3.33
Moda de B: 12
Estadístico t = 3.619 | valor p = 0.0020
Conclusión: la diferencia de promedios es significativa (p < 0.05)
Correlación r = 0.995 | valor p = 0.000003
P(nota <= 12): 0.1587
P(nota >= 17): 0.0668
```

Cómo interpretar

Prueba t: compara los promedios de dos grupos. Como el **valor p = 0.0020** es menor que 0.05, concluimos que la sección A rinde distinto (mejor) que la B, y no es casualidad.
Correlación r = 0.995: valor cercano a 1, es decir, a más horas de estudio, mayor nota (relación positiva muy fuerte).
Distribución normal: si las notas siguen una normal con media 14 y desviación 2, solo el 15.87 % sacaría 12 o menos y el 6.68 % sacaría 17 o más.

Paso 10. Optimización, integración, álgebra lineal e interpolación

Crea **02_scipy_calculo.py**. Estos problemas aparecen constantemente en IA: entrenar un modelo es, en el fondo, **minimizar** una función de error.

Archivo: 02_scipy_calculo.py

```
# 02_scipy_calculo.py - Optimización, integración, álgebra lineal e interpolación
import numpy as np
from scipy import optimize, integrate, linalg, interpolate

# 1. Optimización: encontrar el mínimo de f(x) = (x - 3)² + 2
def f(x):
    return (x - 3) ** 2 + 2

resultado = optimize.minimize_scalar(f)
print(f"Mínimo en x = {resultado.x:.4f}, f(x) = {resultado.fun:.4f}")

# 2. Ajuste de curvas: encontrar a y b en y = a·x + b
x = np.array([0, 1, 2, 3, 4, 5])
y = np.array([1.1, 2.9, 5.2, 7.1, 8.8, 11.2])
def recta(x, a, b):
    return a * x + b
(a, b), _ = optimize.curve_fit(recta, x, y)
print(f"Recta ajustada: y = {a:.3f}x + {b:.3f}")

# 3. Integración numérica: área bajo x² entre 0 y 3
area, error = integrate.quad(lambda t: t ** 2, 0, 3)
print(f"Integral de x² de 0 a 3 = {area:.4f}")

# 4. Álgebra lineal: resolver el sistema
#    2x + 3y = 8
#    3x -  y = 1
A = np.array([[2, 3], [3, -1]])
b_vec = np.array([8, 1])
solucion = linalg.solve(A, b_vec)
print("Solución del sistema (x, y):", solucion)
print("Determinante de A:", round(linalg.det(A), 2))

# 5. Interpolación: estimar valores intermedios (spline cúbico)
horas = np.array([0, 2, 4, 6, 8])
temperatura = np.array([15, 18, 24, 22, 17])
interp = interpolate.CubicSpline(horas, temperatura)
print("Temperatura estimada a las 5 h:", round(float(interp(5)), 2))
```

Resultado esperado en la terminal

```
Mínimo en x = 3.0000, f(x) = 2.0000
Recta ajustada: y = 2.003x + 1.043
Integral de x² de 0 a 3 = 9.0000
Solución del sistema (x, y): [1. 2.]
Determinante de A: -11.0
Temperatura estimada a las 5 h: 23.94
```

Explicación

`minimize_scalar` encontró que la función (x − 3)² + 2 tiene su mínimo en **x = 3**, con valor **2**. `curve_fit` calculó la recta que mejor se ajusta a los puntos: **y ≈ 2.003x + 1.043**. Es la misma idea de la regresión lineal.

> `quad` calculó el área bajo x^2 entre 0 y 3, que es exactamente **9** (comprobable con $x^3/3$).
> `linalg.solve` resolvió el sistema de ecuaciones: **x = 1, y = 2**.
> `CubicSpline` estimó la temperatura a las 5 h (dato que no se midió) en **23.94 °C**.

Parte 3. Procesamiento de lenguaje natural con NLTK

Paso 11. Descargar los recursos de NLTK

NLTK necesita descargar modelos y listas de palabras adicionales. Crea **03_nltk_preparar.py** y ejecútalo **una sola vez** (con Internet).

Archivo: 03_nltk_preparar.py

```
# 03_nltk_preparar.py - Descargar los recursos de NLTK (se ejecuta UNA sola vez)
import nltk

print("Versión de NLTK:", nltk.__version__)
recursos = ["punkt", "punkt_tab", "stopwords"]
for recurso in recursos:
    nltk.download(recurso)
print("Recursos descargados correctamente")
```

Resultado esperado en la terminal

```
Versión de NLTK: 3.9.x
[nltk_data] Downloading package punkt to C:\Users\...\nltk_data...
[nltk_data]   Unzipping tokenizers\punkt.zip.
[nltk_data] Downloading package punkt_tab to C:\Users\...\nltk_data...
[nltk_data] Downloading package stopwords to C:\Users\...\nltk_data...
Recursos descargados correctamente
```

Recurso	Para qué sirve
`punkt` y `punkt_tab`	Modelos para dividir textos en oraciones y palabras (las versiones recientes de NLTK usan `punkt_tab`).
`stopwords`	Listas de palabras vacías (artículos, preposiciones, etc.) en varios idiomas, incluido el español.

Importante

Si aparece **"Resource punkt_tab not found"**, vuelve a ejecutar este archivo. Si tu red bloquea la descarga, pide al instructor la carpeta **nltk_data** y cópiala en `C:\Users\TU_USUARIO\nltk_data`.

Paso 12. Preprocesamiento de texto: del texto a las palabras útiles

Crea **04_nltk_texto.py**. Antes de que un modelo de IA pueda trabajar con texto, el texto debe limpiarse y dividirse. A este proceso se le llama **preprocesamiento**.

Etapa	Técnica	Ejemplo
1. Oraciones	`sent_tokenize`	Un párrafo → lista de oraciones.
2. Tokens	`word_tokenize`	"La IA aprende." → ['La', 'IA', 'aprende', '.']
3. Normalización	`lower()` e `isalpha()`	Pasa a minúsculas y quita signos de puntuación.
4. Palabras vacías	`stopwords.words("spanish")`	Elimina: la, de, en, los, para, está...
5. Stemming	`SnowballStemmer("spanish")`	modelos → model, aprendemos → aprend

Etapa	Técnica	Ejemplo
6. Frecuencias	`FreqDist`, `ngrams`	Palabras más repetidas y pares de palabras.

Archivo: 04_nltk_texto.py

```
# 04_nltk_texto.py - Tokenizar, limpiar, lematizar raíces y contar palabras
import nltk
from nltk.tokenize import sent_tokenize, word_tokenize
from nltk.corpus import stopwords
from nltk.stem import SnowballStemmer
from nltk.probability import FreqDist
from nltk.util import ngrams

texto = ("La inteligencia artificial está transformando la industria. "
         "En SENATI aprendemos Python para crear modelos de inteligencia artificial. "
         "Los modelos aprenden patrones a partir de los datos.")

# 1. Dividir en oraciones
oraciones = sent_tokenize(texto, language="spanish")
print("Número de oraciones:", len(oraciones))
for i, o in enumerate(oraciones, start=1):
    print(f"  {i}. {o}")

# 2. Dividir en palabras (tokens)
tokens = word_tokenize(texto, language="spanish")
print("Primeros 10 tokens:", tokens[:10])

# 3. Normalizar: minúsculas y solo palabras (sin signos de puntuación)
palabras = [t.lower() for t in tokens if t.isalpha()]

# 4. Quitar palabras vacías (stopwords): la, de, en, los, para...
vacias = set(stopwords.words("spanish"))
utiles = [p for p in palabras if p not in vacias]
print("Palabras útiles:", utiles)

# 5. Stemming: reducir cada palabra a su raíz
stemmer = SnowballStemmer("spanish")
raices = [stemmer.stem(p) for p in utiles]
print("Raíces:", raices)

# 6. Frecuencia de palabras
frecuencia = FreqDist(utiles)
print("Palabras más frecuentes:", frecuencia.most_common(3))

# 7. Bigramas (pares de palabras consecutivas)
bigramas = list(ngrams(utiles, 2))
print("Primeros 3 bigramas:", bigramas[:3])
```

Resultado esperado en la terminal

```
Número de oraciones: 3
  1. La inteligencia artificial está transformando la industria.
  2. En SENATI aprendemos Python para crear modelos de inteligencia artificial.
  3. Los modelos aprenden patrones a partir de los datos.
Primeros 10 tokens: ['La', 'inteligencia', 'artificial', 'está', 'transformando', 'la',
'industria', '.', 'En', 'SENATI']
```

```
Palabras útiles: ['inteligencia', 'artificial', 'transformando', 'industria', 'senati',
'aprendemos', 'python', 'crear', 'modelos', 'inteligencia', 'artificial', 'modelos',
'aprenden', 'patrones', 'partir', 'datos']
Raíces: ['inteligent', 'artificial', 'transform', 'industri', 'senati', 'aprend', 'python',
'cre', 'model', 'inteligent', 'artificial', 'model', 'aprend', 'patron', 'part', 'dat']
Palabras más frecuentes: [('inteligencia', 2), ('artificial', 2), ('modelos', 2)]
Primeros 3 bigramas: [('inteligencia', 'artificial'), ('artificial', 'transformando'),
('transformando', 'industria')]
(Las raíces pueden variar ligeramente según la versión de NLTK)
```

Explicación

Al quitar las palabras vacías quedan solo las palabras con significado, que son las que interesan al modelo.
El **stemming** une palabras de la misma familia: *aprendemos* y *aprenden* quedan como **aprend**, por eso el modelo las cuenta como una sola idea.
Para ver la frecuencia en un gráfico agrega al final `frecuencia.plot(10)`: VS Code abrirá una ventana con la gráfica de las 10 palabras más frecuentes.

Paso 13. Clasificador de opiniones (análisis de sentimiento)

Crea **05_nltk_sentimiento.py**. Entrenaremos un clasificador **Naive Bayes** que aprende qué palabras suelen aparecer en opiniones positivas y cuáles en negativas.

Archivo: 05_nltk_sentimiento.py

```
# 05_nltk_sentimiento.py - Clasificador de opiniones con Naive Bayes de NLTK
import nltk
from nltk.tokenize import word_tokenize
from nltk.corpus import stopwords

vacias = set(stopwords.words("spanish"))

def extraer_caracteristicas(frase):
    """Convierte una frase en un diccionario {palabra: True}."""
    tokens = word_tokenize(frase.lower(), language="spanish")
    return {t: True for t in tokens if t.isalpha() and t not in vacias}

# Datos de entrenamiento: opiniones de estudiantes sobre un taller
datos = [
    ("El taller fue excelente y muy práctico", "positivo"),
    ("Me gustó mucho, el instructor explica muy bien", "positivo"),
    ("Aprendí bastante, las prácticas son útiles", "positivo"),
    ("Excelente contenido, muy recomendable", "positivo"),
    ("El curso es interesante y divertido", "positivo"),
    ("El taller fue aburrido y muy largo", "negativo"),
    ("No me gustó, la explicación fue confusa", "negativo"),
    ("Las prácticas son difíciles y poco útiles", "negativo"),
    ("Pésimo contenido, no lo recomiendo", "negativo"),
    ("El curso es lento y aburrido", "negativo"),
]

conjunto = [(extraer_caracteristicas(frase), etiqueta) for frase, etiqueta in datos]

# Entrenar el clasificador
clasificador = nltk.NaiveBayesClassifier.train(conjunto)

# Probar con frases nuevas
pruebas = [
    "Un taller muy práctico y excelente",
    "La clase fue aburrida y confusa",
    "Recomendable, el curso es divertido y práctico",
]
for frase in pruebas:
    resultado = clasificador.classify(extraer_caracteristicas(frase))
    print(f"'{frase}' -> {resultado}")

print("Exactitud en entrenamiento:", nltk.classify.accuracy(clasificador, conjunto))
clasificador.show_most_informative_features(5)
```

Resultado esperado (puede variar ligeramente)

```
'Un taller muy práctico y excelente' -> positivo
'La clase fue aburrida y confusa' -> negativo
'Recomendable, el curso es divertido y práctico' -> positivo
Exactitud en entrenamiento: 1.0
Most Informative Features
```

```
            (lista de palabras con su relación positivo:negativo, por ejemplo aburrido =
True ...)
```

Explicación

Cada frase se convierte en un diccionario de características, por ejemplo `{'taller': True, 'excelente': True, 'práctico': True}`.
`show_most_informative_features` muestra las palabras que más ayudan a decidir: una línea como **aburrido = True negati : positi = 2.3 : 1.0** significa que esa palabra es más frecuente en opiniones negativas.
Con solo 10 ejemplos el modelo puede equivocarse con palabras que nunca vio (por ejemplo, *aburrida* en femenino). Por eso los modelos reales se entrenan con miles de ejemplos y se aplica stemming.

Parte 4. Proyecto integrador: SciPy + NLTK

Paso 14. Medir la similitud entre textos

Crea **06_scipy_nltk_similitud.py**. Los buscadores y chatbots necesitan saber qué tan parecidos son dos textos. Usaremos **NLTK** para preparar los textos, **NumPy** para convertirlos en vectores (bolsa de palabras) y **SciPy** para calcular la **similitud del coseno**.

Archivo: 06_scipy_nltk_similitud.py

```
# 06_scipy_nltk_similitud.py - Proyecto integrador: similitud entre textos
import numpy as np
from nltk.tokenize import word_tokenize
from nltk.corpus import stopwords
from nltk.stem import SnowballStemmer
from scipy.spatial.distance import cosine

vacias = set(stopwords.words("spanish"))
stemmer = SnowballStemmer("spanish")

def procesar(texto):
    """NLTK: tokeniza, quita stopwords y obtiene raíces."""
    tokens = word_tokenize(texto.lower(), language="spanish")
    return [stemmer.stem(t) for t in tokens if t.isalpha() and t not in vacias]

documentos = {
    "doc1": "Los modelos de inteligencia artificial aprenden de los datos.",
    "doc2": "La inteligencia artificial usa datos para entrenar modelos.",
    "doc3": "El mantenimiento de motores eléctricos requiere herramientas.",
}

procesados = {nombre: procesar(t) for nombre, t in documentos.items()}
vocabulario = sorted(set(p for lista in procesados.values() for p in lista))
print("Vocabulario:", vocabulario)

# Vector de conteo (bolsa de palabras) para cada documento
def vectorizar(lista):
    return np.array([lista.count(p) for p in vocabulario])

vectores = {nombre: vectorizar(lista) for nombre, lista in procesados.items()}
```

```
# SciPy: similitud del coseno = 1 - distancia del coseno
def similitud(a, b):
    return 1 - cosine(vectores[a], vectores[b])

print(f"Similitud doc1 - doc2: {similitud('doc1', 'doc2'):.3f}")
print(f"Similitud doc1 - doc3: {similitud('doc1', 'doc3'):.3f}")
print(f"Similitud doc2 - doc3: {similitud('doc2', 'doc3'):.3f}")
```

Resultado esperado (puede variar ligeramente)

```
Vocabulario: ['aprend', 'artificial', 'dat', 'entren', ...]   (lista de raíces ordenadas)
Similitud doc1 - doc2: 0.73 aprox.   (textos del mismo tema -> valor alto)
Similitud doc1 - doc3: 0.000         (temas distintos -> sin palabras en común)
Similitud doc2 - doc3: 0.000
```

Explicación

La **bolsa de palabras** representa cada texto como un vector con cuántas veces aparece cada raíz del vocabulario.
La **similitud del coseno** va de 0 (nada en común) a 1 (idénticos). doc1 y doc2 hablan de IA y comparten 4 raíces, por eso su similitud es alta; doc3 trata de motores eléctricos y su similitud es 0.
Este es el principio básico de los buscadores de documentos y de los sistemas de preguntas frecuentes de un chatbot.

Usar el depurador de VS Code

El depurador te permite detener el programa en una línea y revisar el valor de cada variable. Es la mejor forma de entender qué hace el código paso a paso.

1. Haz clic a la izquierda del número de línea donde quieres detenerte: aparecerá un **punto rojo** (punto de interrupción). También puedes usar `F9`.
2. Presiona `F5` y elige **Python Debugger > Python File** (archivo actual).
3. El programa se detiene en el punto rojo. En el panel **VARIABLES** (izquierda) verás los arreglos, tensores o DataFrames con sus valores.
4. Usa `F10` para avanzar línea por línea, `F11` para entrar a una función y `Shift + F5` para detener.
5. En la **CONSOLA DE DEPURACIÓN** puedes escribir expresiones, por ejemplo `df.head()`, y ver el resultado en ese momento.

Sugerencia

Coloca un punto de interrupción en la línea `vectores = ...` del Paso 14 y revisa en el panel **VARIABLES** el contenido de `procesados` y `vocabulario`.

Parte 5. Errores frecuentes y cómo resolverlos

Mensaje de error	Causa	Solución
`No se encontró Python; ejecutar sin argumentos para instalar desde Microsoft Store...`	Python no está instalado o no está en el PATH; Windows redirige el comando a la Microsoft Store.	Instala Python desde python.org marcando **Add python.exe to PATH**. Desactiva los alias en **Configuración > Aplicaciones > Configuración avanzada de aplicaciones > Alias de ejecución de aplicaciones** (python.exe y python3.exe). Cierra y vuelve a abrir VS Code.
`LookupError: Resource punkt_tab not found`	No se descargaron los recursos de NLTK.	Ejecuta el archivo del Paso 11 o `nltk.download("punkt_tab")`.
`LookupError: Resource stopwords not found`	Falta la lista de palabras vacías.	Ejecuta `nltk.download("stopwords")`.
`ModuleNotFoundError: No module named 'scipy'`	SciPy no está en el entorno activo.	Activa **venv** y ejecuta `pip install scipy`.
`LinAlgError: Singular matrix`	El sistema de ecuaciones no tiene solución única.	Revisa los coeficientes; el determinante no debe ser 0.
`RuntimeWarning: invalid value encountered` en `cosine`	Uno de los vectores está lleno de ceros.	Verifica que el texto tenga palabras después de quitar las stopwords.
Tildes o eñes que se ven mal en la terminal	Codificación del archivo distinta de UTF-8.	En la barra de estado de VS Code haz clic en la codificación y elige **UTF-8**.

Parte 6. Ejercicios propuestos

1. **SciPy:** registra las notas de dos de tus cursos (al menos 8 notas por curso) y aplica `ttest_ind`. ¿Hay diferencia significativa entre ambos cursos?
2. **SciPy:** usa `optimize.minimize_scalar` para hallar el mínimo de f(x) = x² − 6x + 13 y comprueba el resultado derivando a mano.
3. **SciPy:** resuelve con `linalg.solve` el sistema: x + y + z = 6, 2x − y + z = 3, x + 2y − z = 2.
4. **NLTK:** copia un párrafo de una noticia en español, obtén las 10 palabras más frecuentes (sin stopwords) y muéstralas en un gráfico con `frecuencia.plot(10)`.
5. **NLTK:** agrega 10 opiniones más al clasificador del Paso 13 (5 positivas y 5 negativas) y aplica stemming en `extraer_caracteristicas`. ¿Mejora la clasificación de “La clase fue aburrida”?
6. **Reto integrador:** agrega dos documentos más al Paso 14 y, para una pregunta escrita por el usuario con `input()`, muestra cuál documento es el más parecido.

Lista de cotejo (autoevaluación)

Criterio	Logrado	En proceso
Instalé SciPy y NLTK y descargué los recursos de NLTK.	☐	☐
Apliqué estadística descriptiva, prueba t, correlación y distribución normal.	☐	☐
Resolví optimización, ajuste de curvas, integrales, sistemas e interpolación.	☐	☐
Tokenicé un texto, quité stopwords y obtuve raíces con stemming.	☐	☐
Calculé frecuencias y bigramas de un texto.	☐	☐
Entrené y probé un clasificador de opiniones con NLTK.	☐	☐
Calculé la similitud entre textos combinando NLTK y SciPy.	☐	☐
Usé el depurador de VS Code para revisar variables.	☐	☐